

Manipulação *de arquivos I*

Guardar dados num arquivo é o que faz um programa lembrar das coisas depois de fechado. Nesta primeira aula da série, os arquivos de **texto**: abrir com `open()` dentro de um `with`, os modos `"r"`, `"w"`, `"a"` e `"x"`, ler tudo, linha a linha ou numa lista, escrever com `write` e `print(file=)`, a importância do `encoding="utf-8"` e os erros mais comuns.

`open() + with`

`"r" · "w" · "a" · "x"`

`read · readline · readlines`

`for linha in arq`

`write · print(file=)`

`encoding="utf-8"`

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Reading and Writing Files; Built-in Functions: open; io; Built-in Exceptions: FileNotFoundError, FileExistsError), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Criando um arquivo *com with*

escrever.py

```
with open("lista.txt", "w", encoding="utf-8") as arq:
    arq.write("café\n")          # write não põe o \n sozinho
    arq.write("pão\n")
    arq.writelines(["leite\n", "maçã\n"])

print("arquivo gravado")
print(arq.closed)              # o with já fechou
```

SAÍDA

```
arquivo gravado
True
```

- ✓ `open(nome, "w")` cria o arquivo ou **apaga** o conteúdo de um que já existe
- ✓ `write` grava exatamente o texto dado: a quebra de linha é com você (`\n`)
- ✓ O `with` fecha o arquivo no fim do bloco, mesmo se der erro no meio

Sempre `encoding="utf-8"`: sem ele, o Python usa o padrão do sistema, e acentos podem sair trocados em outro computador.

read, readline e readlines

ler.py

```
with open("lista.txt", encoding="utf-8") as arq: # "r": padrão
    tudo = arq.read()
    print(repr(tudo))

with open("lista.txt", encoding="utf-8") as arq:
    print(arq.readline().strip()) # uma linha
    print(arq.readlines())       # o resto, numa lista
```

SAÍDA

```
'café\npão\nleite\nmaçã\n'
café
['pão\n', 'leite\n', 'maçã\n']
```

MÉTODO

DEVOLVE

`read()`

o arquivo inteiro numa string

`readline()`

a próxima linha (com o `\n`)

`readlines()`

as linhas restantes numa lista

O arquivo guarda a posição de leitura: depois do `readline()`, o `readlines()` começa da segunda linha.

O arquivo é *um iterador*

percorrer.py

```
with open("lista.txt", encoding="utf-8") as arq:  
    for numero, linha in enumerate(arq, start=1):  
        print(numero, linha.rstrip("\n"))
```

SAÍDA

```
1 café  
2 pão  
3 leite  
4 maçã
```

- ✓ `for linha in arq` lê uma linha por vez: funciona com arquivos de qualquer tamanho (aula 05)
- ✓ Cada linha vem com o `\n` do fim: `rstrip("\n")` ou `strip()` tira
- ✓ `enumerate(arq, start=1)` numera as linhas (aula 15)

É o jeito preferido de ler texto: simples e econômico em memória.

Modo "a": *sem apagar o que existe*

acrescentar.py

```
with open("diario.txt", "w", encoding="utf-8") as arq:
    arq.write("segunda: estudei iteradores\n")

with open("diario.txt", "a", encoding="utf-8") as arq: # "a"
    arq.write("terça: estudei geradores\n")
    print("quarta: estudei regex", file=arq) # print grava

with open("diario.txt", encoding="utf-8") as arq:
    print(arq.read(), end="")
```

SAÍDA

```
segunda: estudei iteradores
terça: estudei geradores
quarta: estudei regex
```

- ✓ "a" (append) escreve no fim e mantém o conteúdo anterior
- ✓ Ideal para logs e diários: cada execução acrescenta linhas
- ✓ `print(..., file=arq)` grava no arquivo e já põe o `\n`

O exemplo começa com "w" só para sair igual a cada execução.

Qual modo *usar*?

modos.py

```
import os

if os.path.exists("unico.txt"):
    os.remove("unico.txt")      # para o exemplo recomeçar

with open("unico.txt", "x", encoding="utf-8") as arq: # cria
    arq.write("primeira vez\n")
print("criado")

try:
    open("unico.txt", "x", encoding="utf-8") # já existe
except FileExistsError as erro:
    print("Erro:", erro)
```

SAÍDA

```
criado
Erro: [Errno 17] File exists: 'unico.txt'
```

MODOS	FAZ	SE O ARQUIVO JÁ EXISTE
"r"	lê (padrão)	lê; se não existe, erro
"w"	escreve	apaga e recomeça
"a"	acrescenta	escreve no fim
"x"	cria	dá <code>FileExistsError</code>
+ "b"	binário (aula 19)	bytes em vez de texto
"w" apaga sem perguntar. Para não sobrescrever um arquivo importante por engano, use "x" .		

Arquivo que não existe *e bytes de verdade*

erros.py

```
try:
    with open("nao_existe.txt", encoding="utf-8") as arq:
        print(arq.read())
except FileNotFoundError:
    print("Arquivo não encontrado: confira o nome e a pasta")

with open("lista.txt", "rb") as arq:      # os bytes de verdade
    print(arq.read(8))
```

SAÍDA

```
Arquivo não encontrado: confira o nome e a pasta
b'caf\xc3\xa9\np\xc3'
```

- ✓ Ler um arquivo que não existe lança `FileNotFoundError`: trate e mostre uma mensagem útil
- ✓ O caminho é relativo à pasta de onde o programa roda, não à pasta do `.py`
- ✓ Em UTF-8, "é" e "ã" ocupam dois bytes cada (`\xc3\xa9`): é o que o `encoding` traduz

Palavras *mais frequentes*

Enunciado: grave um texto num arquivo, leia de volta linha a linha e mostre as três palavras mais frequentes e o total de palavras.

Uma resposta possível

```
from collections import Counter
import re

with open("texto.txt", "w", encoding="utf-8") as arq:
    arq.write("O sol nasce. O sol se põe.\n")
    arq.write("A lua nasce depois do sol.\n")

palavras = Counter()
with open("texto.txt", encoding="utf-8") as arq:
    for linha in arq:
        palavras.update(re.findall(r"\w+", linha.lower()))

print(palavras.most_common(3))
print(sum(palavras.values()), "palavras no total")
```

SAÍDA

```
[('sol', 3), ('o', 2), ('nasce', 2)]
13 palavras no total
```

- ✓ `re.findall(r"\w+", ...)` separa as palavras sem pontuação (aula 06)
- ✓ `Counter.update` soma as contagens de cada linha
- ✓ Desafio: ignore palavras curtas como "o", "a", "se"

Teste o que *ficou*

1. Abrir com "w" um arquivo que já tem conteúdo:

A) Acrescenta no fim

B) Apaga e recomeça

C) Dá erro

D) Só lê

B.

2. Para não esquecer de fechar o arquivo:

A) Use with

B) Use "a"

C) Use readlines()

D) Não precisa fechar

A.

3. Para ler um arquivo enorme sem lotar a memória:

A) `arq.read()`

B) `arq.readlines()`

C) `for linha in arq`

D) `print(arq)`

C.

4. Qual modo dá erro se o arquivo já existir?

A) "r"

B) "w"

C) "a"

D) "x"

D.

Texto que fica *depois do programa*

Abra arquivos com `with open(nome, modo, encoding="utf-8")`. `"w"` recria, `"a"` acrescenta, `"x"` só cria e `"r"` lê. Leia linha a linha com `for`, tire o `\n` com `strip`, escreva com `write` ou `print(file=)` e trate o `FileNotFoundError`.

`with open()`

`r · w · a · x`

`for linha in arq`

`write · print(file=)`

`utf-8`

Próxima aula → 17 · Manipulação de Arquivos II

Fontes: docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files; docs.python.org/3/library/functions.html#open; docs.python.org/3/library/io.html; docs.python.org/3/library/exceptions.html (`FileNotFoundError`, `FileExistsError`); docs.python.org/3/library/collections.html#collections.Counter, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.